

Operating System Revision: Remaining Slides

LECTURE 6A: User & OS Interface

1. User Interactions

- Command-Line Interface (CLI)
 - Text-based input (e.g., `rm file.txt`)
 - Preferred by system admins/advanced users for speed & scripting
- Graphical User Interface (GUI)
 - Visual interface using icons, windows, touchscreens
 - Favored by general users for ease of use

2. Shell (Command Interpreter)

- Acts as the intermediary that interprets commands.
- Examples in UNIX/Linux: Bourne Shell, C Shell, Korn Shell.
- Most commands are external programs, not built into the shell.

3. How Commands Execute

- **CLI Example:**
 - Typing `rm file.txt` → Shell locates `rm` program, loads into memory, and runs it with `file.txt` as input.
 - **GUI Example:**
 - Interaction through menus/icons (e.g., file deletion via right-click).
-

LECTURE 6B: System Calls & APIs

1. What is a System Call?

- **Definition:** A programmatic request from an application to the OS kernel.
- **Key Role:** Bridge between user programs and hardware.
- **Entry Point:** The *only* way to enter kernel mode.

2. How System Calls Work

1. Application runs in user mode.
2. System call is made (via API).
3. CPU switches to kernel mode.
4. OS executes the request (e.g., file I/O, process creation).

5. Control returns to user mode.

3. Passing Parameters

- **Methods:** Registers (fast, limited), Memory Block/Table (common in Linux), or the Stack (flexible).

4. System Call Types (with Examples)

- **Process Control:** `fork()` , `exec()` , `exit()` , `wait()`
- **File Management:** Create, delete, open, close, read, write
- **Device Management:** Request/release devices, device-specific operations
- **Information Maintenance:** Get/set system data (e.g., time, attributes)
- **Communication:** IPC via message passing or shared memory

5. APIs

- Provide a simpler interface to system calls.
 - Examples: Windows API, POSIX API, Java API.
 - **Benefits:** Portability, simplicity, and standardization.
-

LECTURE 9 & 10: Processes & Threads

1. What is a Process?

- **Definition:** A program in execution (active), unlike a stored program (passive).
- **Lifecycle States:**
 - **Start:** Process created
 - **Ready:** Waiting in line for the CPU
 - **Running:** Actively executing
 - **Waiting:** Blocked (e.g., for I/O)
 - **Terminated:** Completed execution

2. Process Components

- **Process Control Block (PCB)**
 - Stores process state, PID, CPU registers, program counter, scheduling, memory, I/O, and accounting information.
- **Threads**
 - Smallest execution unit.
 - A process can contain multiple threads to perform tasks concurrently.
 - **Real-Life Analogy:** Multiple workers (threads) in a single team (process) sharing resources.

3. Process Scheduling & Context Switching

- **Schedulers:**
 - **Long-Term (Job Scheduler):** Loads jobs into memory.
 - **Short-Term (CPU Scheduler):** Selects which process gets CPU next.
 - **Context Switch:** Saving the state of the current process and loading another's state—this is overhead but essential for multitasking.
 - **Interprocess Communication (IPC)**
 - **Models:** Shared Memory vs. Message Passing.
 - **Example Problem:** Producer-Consumer scenario (synchronization in shared buffers).
-

LECTURE 11: Process Synchronization

1. Synchronization Concepts

- **Race Condition:** When processes access shared data concurrently, leading to unpredictable results.
- **Critical Section:** Part of a process where shared resources are accessed. Only one process should be in here at any given time.

2. Requirements for a Critical Section Solution

- **Mutual Exclusion:** One process at a time.
- **Progress:** A process not in a critical section should not block others from entering when possible.
- **Bounded Waiting:** Every process must eventually get a turn.

3. Producer-Consumer Problem

- **Scenario:** Producer adds items to a fixed buffer; Consumer removes items.
 - **Buffer Types:**
 - **Unbounded:** Infinite size.
 - **Bounded:** Fixed size → must wait when full or empty.
 - **Synchronization:** Use a count variable to keep track of items.

4. Peterson's Solution (for 2 Processes)

- **Concept Recap:** Processes take turns entering the critical section using shared flags and a turn variable.
- *(See your earlier Peterson's method revision for details with easy variable names such as `wantsToGo` and `whosTurn`.)*

LECTURE 12: Mutex, Semaphores & Deadlocks

1. Mutex Lock

- **Definition:** A MUTual EXclusion object that allows only one thread to use a resource at a time.
- **Usage:**
 - **Acquire/Lock:** Before entering critical region.
 - **Release/Unlock:** After finishing the critical region.

2. Semaphores (Dijkstra)

- **Types:**
 - **Counting Semaphore:** Keeps an integer count (for multiple instances).
 - **Binary Semaphore:** Value is 0 or 1 (similar to a mutex).
- **Operations:**
 - **WAIT (P Operation):** Decrement the semaphore. If ≤ 0 , process waits.
 - **SIGNAL (V Operation):** Increment the semaphore, potentially waking waiting processes.

3. Deadlocks

- **Definition:** Situations where processes are stuck waiting indefinitely for each other.
- **Necessary Conditions:**

1. **Mutual Exclusion:** A resource is held exclusively.
 2. **Hold and Wait:** Processes are holding resources while waiting for others.
 3. **No Preemption:** Resources are released only voluntarily.
 4. **Circular Wait:** A closed chain of processes exists where each process holds at least one resource needed by the next.
- **Resource Allocation Graph (RAG)**
 - **Nodes:**
 - **Circles:** Processes
 - **Squares:** Resources (with dots showing instances)
 - **Edges:**
 - **Request Edge:** From process to resource.
 - **Assignment Edge:** From resource to process.
 - **Cycle Detection:**
 - **No Cycle:** No deadlock.
 - **Cycle** with one instance per resource → definite deadlock.
 - **Cycle** with multiple instances → possible deadlock.